

Code-Layout, Readability and Reusability

Date	30 June 2026
Team ID	
Project Name	Rising Waters
Maximum Marks	5 Marks

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Wellorganized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist

#	Code Quality Parameter	Followed	Remarks
1	Consistent Indentation	Yes	Verified via linter runs.
2	Proper File Structure	Yes	Segregated app, models, and training.
3	Meaningful Variable Names	Yes	Variables reflect indicators (e.g., life_exp).
4	Function / Method Names	Yes	Descriptive action names (e.g., init_db).
5	Code Comments	Yes	Docstrings explain methods.
6	Modular Design	Yes	Separated backend REST and UI layers.
7	No Redundant Code	Yes	Shared template structures and style sheets.
8	Error Handling	Yes	Try-except blocks catch prediction errors.

Reusable Components / Modules

#	Component / Module Name	Language / Technology	Where Reused	Reusability
1	SQLAlchemy Database Session Manager	Python (app.py, models.py)	All DB read/write routes	High
2	AJAX POST Request Wrapper	JavaScript (index.js)	All workspace API calls	High
3	SVG Visual Report Builder	Python (app.py)	Report generation endpoint	High
4	Missing Data Mean Imputation	Python (train.py, app.py)	Training and inference pipelines	High
5	Glassmorphism Card Layout Styles	CSS (index.css)	All dashboard panels	High

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	PEP 8 compliant codebase; clean separation of roles.
Readability	5	Descriptive variable names and complete comments.
Reusability	5	Abstracted styles and modular AJAX elements.
Documentation / Comments	5	Thorough documentation and docstrings.
Overall Score	5	Professional, standard-compliant project layout.